

# Inside the Deal Intelligence Agent

How Recall, Reflect, and Retain turn a CRM into organizational memory

Anushka Gorla · Hyderabad, Telangana, India · 29 Sept 2026

Most CRMs are good at one thing: recording what is happening in a deal right now — stage, value, owner, next meeting. What they rarely capture is the thing that actually decides whether a deal closes: the pattern behind it. Has this pricing objection come up before? What did the team do last time a competitor was mentioned at this stage? Which strategy actually worked?

The Deal Intelligence Agent is built to answer exactly that class of question. It sits above the CRM as an intelligence layer, combining Retrieval-Augmented Generation (RAG) over company documents with a three-stage memory architecture — Recall, Reflect, and Retain — that treats every closed deal as a lesson the organization can reuse.

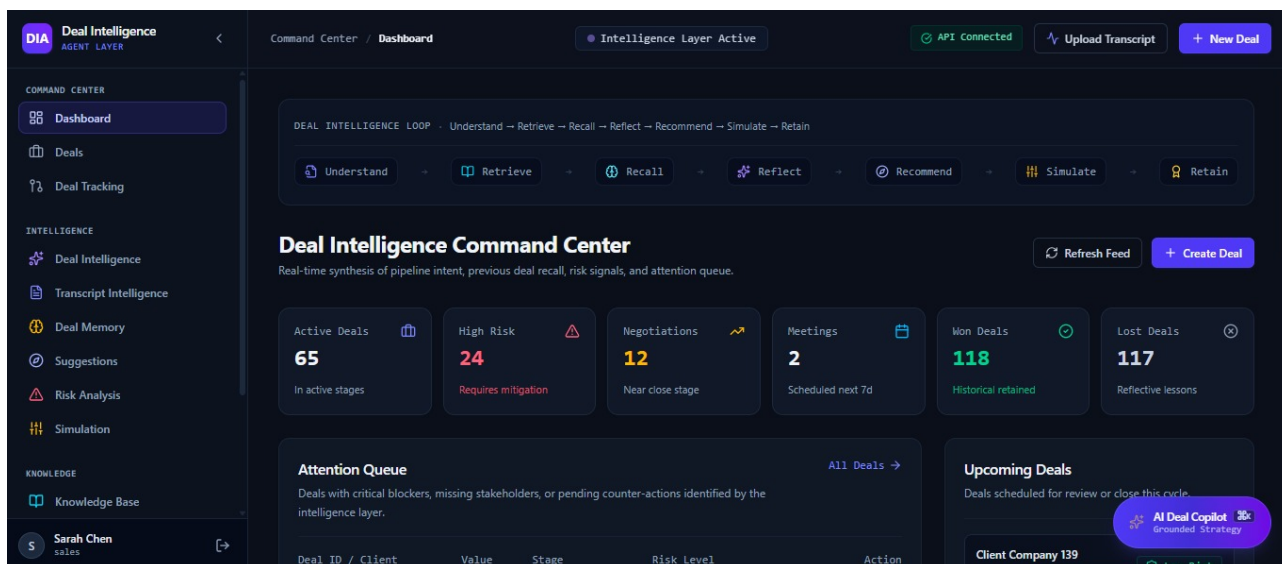


Figure 1 — The Command Center: live pipeline health, risk signals, and the agent's Understand → Retrieve → Recall → Reflect → Recommend → Simulate → Retain loop, always visible at the top of the workspace.

The dashboard makes the agent loop a first-class citizen of the interface rather than something hidden behind an API call. Every deal on the board has moved through some part of that loop, and the loop itself is the organizing idea behind the rest of the product.

## 01 — How the Pieces Fit Together

Underneath the interface, the system is a fairly conventional four-layer stack — a React front end, an Express gateway for CRUD and auth, a FastAPI service running the LangGraph agent, and a data layer combining PostgreSQL, pgvector, and a dedicated memory store. What makes it distinctive is not any single layer; it's how deliberately Recall, Reflect, and Retain are kept as separate, auditable steps rather than one blended “memory” feature.

## Deal Intelligence Agent — System Architecture

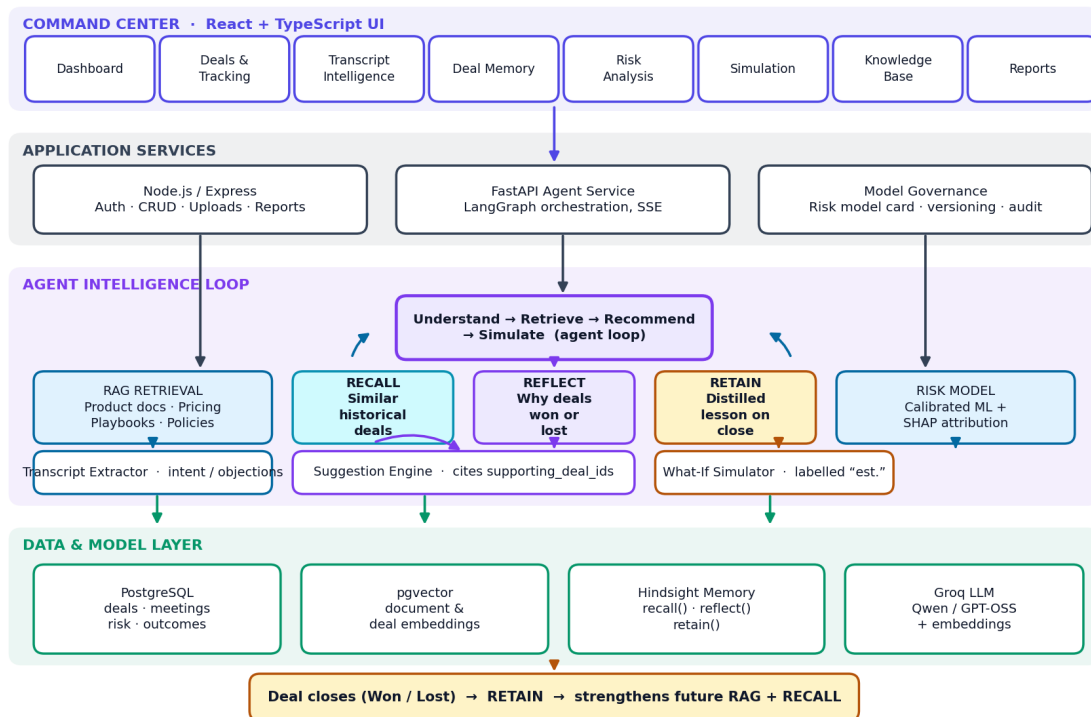


Figure 2 — System architecture: the Command Center (React + TypeScript) talks to an Express API and a FastAPI/LangGraph agent service; the agent intelligence loop — RAG retrieval, Recall, Reflect, Retain, and the risk model — sits over a PostgreSQL + pgvector data layer, with every closed deal feeding back into future Recall and RAG.

That same discipline shows up in how the agent's answers are shaped before they ever reach a rep. Every response the orchestrator produces is typed and structured — not a free-floating chat reply, but a package that carries its own evidence:

```

File explorer | metadata.json | README.md | chatIntelligenceService.ts
├── datasets
├── docs
├── models
├── python-servi...
├── server
│   ├── db
│   ├── middleware
│   ├── routes
│   ├── scripts
│   └── services
│       ├── chatIntell...
│       ├── intelligen...
│       └── pythonM...
├── types
├── src
│   ├── components
│   ├── hooks
│   ├── lib
│   ├── pages
│   └── store
└── ...

chatIntelligenceService.ts
4 import { Extension } from 'langchain';
5 export interface ChatMessageInput {
6   role: 'user' | 'assistant' | 'model';
7   content: string;
8 }
9
10 export interface ChatIntelligenceResult {
11   reply: string;
12   citations: Citation[];
13   suggestedPrompts: string[];
14   confidence: number;
15   dealContext?: {
16     id: string;
17     client: string;
18     stage: string;
19     value: number;
20     risk: string;
21     winProbability?: number;
22   };
23   modelUsed: string;
24 }
25
26 export class ChatIntelligenceService {
27   private static aiClient: GoogleGenAI | null = null;
28
29   private static getAi(): GoogleGenAI | null {
30     const key = process.env.GEMINI_API_KEY;
31     if (!this.aiClient && key && key !== 'MY_GEMINI_API_KEY' && key.trim().length > 10) {
32       this.aiClient = new GoogleGenAI({
33         apiKey: key

```

Figure 3 — The agent's response contract: every reply carries its citations, a confidence score, the deal context it was grounded in, and which model produced it — so downstream UI can never show a claim without its evidence.

Notice what's in that shape: citations, a confidence score, the deal context the answer was grounded in, and which model produced it. None of that is optional metadata bolted on afterward

— it's the contract every part of the UI is built against, which is what makes it possible to show evidence next to every claim instead of asking a rep to trust a paragraph of prose.

## 02 — Recall, Reflect, and Retain, in Practice

The clearest way to see the architecture is to walk through what each stage actually does inside the product, using the Deal Memory workspace as the example.

### Recall: has this happened before?

Recall is the first layer of Deal Memory. Given the current opportunity, it runs a search over historical deals — not a plain keyword match, but a combination of structured filters (industry, deal size, product) and semantic similarity over objections and requirements. The goal is to surface genuine precedent, not merely text that looks alike.

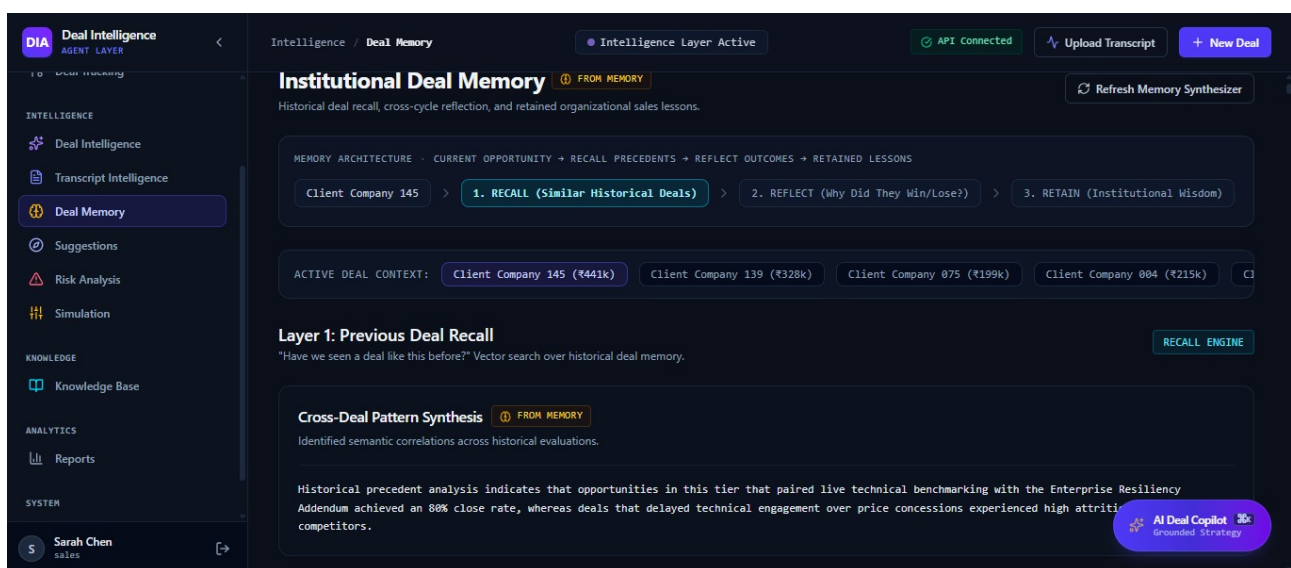


Figure 4 — Institutional Deal Memory: the current opportunity moves through Recall precedents, Reflect on outcomes, and Retain institutional wisdom, with every synthesis explicitly marked “from memory.”

Notice the label on the synthesis card: “From Memory.” That distinction matters more than it looks. It tells the rep this insight came from retained, validated experience — not from the model improvising in the moment. The synthesis itself is also concrete: it names a specific combination (live technical benchmarking paired with a resiliency addendum) and a specific result (an 80% close rate), rather than a vague summary of “similar deals.”

### Reflect: why did it actually go that way?

Reflect takes the deals Recall surfaces and asks a harder question: not just what happened, but why. It compares the strategies used across similar deals, separates what worked from what failed, and produces a short, evidence-cited argument rather than a plain list of past outcomes. In the product, this becomes the basis for the Suggestions view — every recommended next action links back to the specific deals that support it, so a rep can check the reasoning rather than take it on faith.

### Retain: what is actually worth remembering?

Retain runs once, at the moment a deal is marked won or lost. Rather than storing the full transcript history, it writes a distilled record: the objection that mattered, the strategy that was

tried, and the reason for the outcome. That single design choice — write a lesson, not a log — is what keeps the memory layer useful instead of turning into a second, noisier copy of the CRM.

RAG answers what the documents say. Recall answers whether this has happened before. Reflect answers what that history actually means. Retain decides what becomes permanent.

## 03 — Turning Memory into Visible, Defensible Signals

Recall and Reflect are only useful if their output is trustworthy enough to act on. This is where the product's Risk Analysis view earns its place, because a risk score is exactly the kind of output people either believe blindly or ignore entirely. The design choice here is to never show a bare score: a “Medium — 57%” read is paired with the specific signals driving it — historical industry conversion rate, objection patterns, pipeline velocity — each with a stated impact weight. A model-confidence figure sits next to the score too, which is a quiet but important admission: the system is telling the rep how much to trust the number, not just what the number is.

### Exploring options without overstating certainty

The most exposed part of any system like this is the what-if simulator, because it is the easiest place to accidentally imply a forecast the data cannot support. A discount slider that outputs a single confident number is more persuasive than it should be.

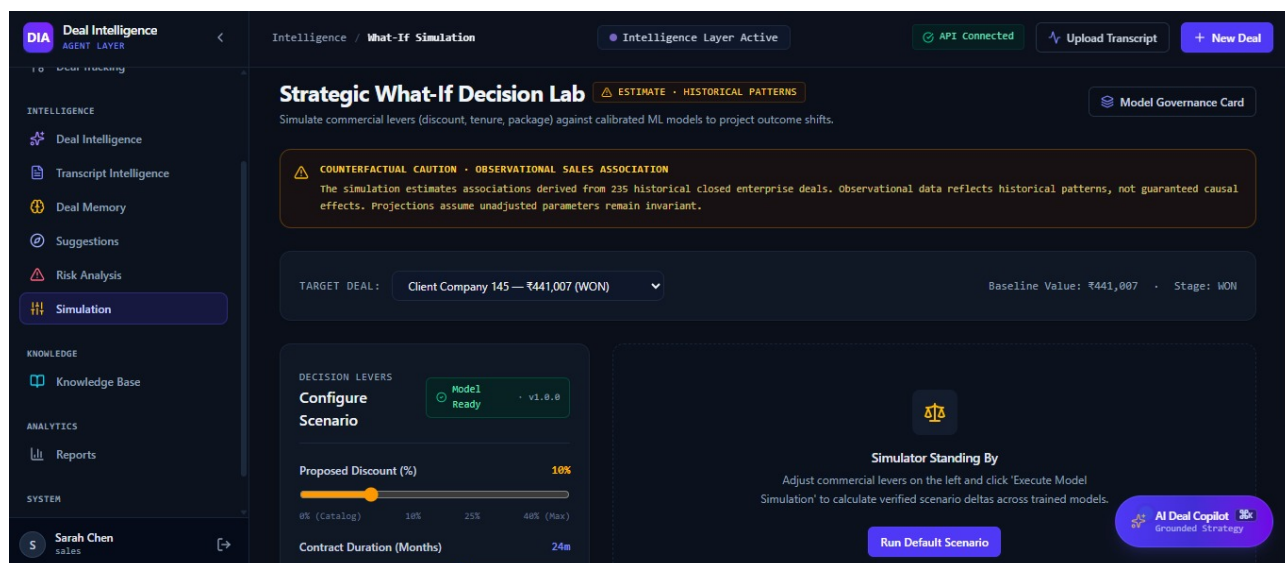


Figure 5 — Strategic What-If Decision Lab: commercial levers compared against historical patterns, with an explicit “estimate” and “observational, not causal” framing built into the interface.

The interface handles this with two visible guardrails. First, every scenario is explicitly labelled “estimate · historical patterns,” not a prediction. Second, a standing caution banner states plainly that the underlying associations are observational, derived from a fixed set of historical closed deals, and do not imply a guaranteed causal effect. That is an unusual thing to put front and center in a product built to sell confidence, and it is exactly the discipline that keeps the simulator credible rather than merely persuasive.

## 04 — Where the Loop Begins: Understanding a Conversation

None of Recall, Reflect, or Retain has anything to reason over until a transcript has actually been read. That is the job of the ingestion pipeline shown below: a rep drops in a call recording, meeting note, or email thread, and the agent parses the dialogue, extracts intent and sentiment, grounds it against company knowledge, and only then hands the result to Recall for synthesis.

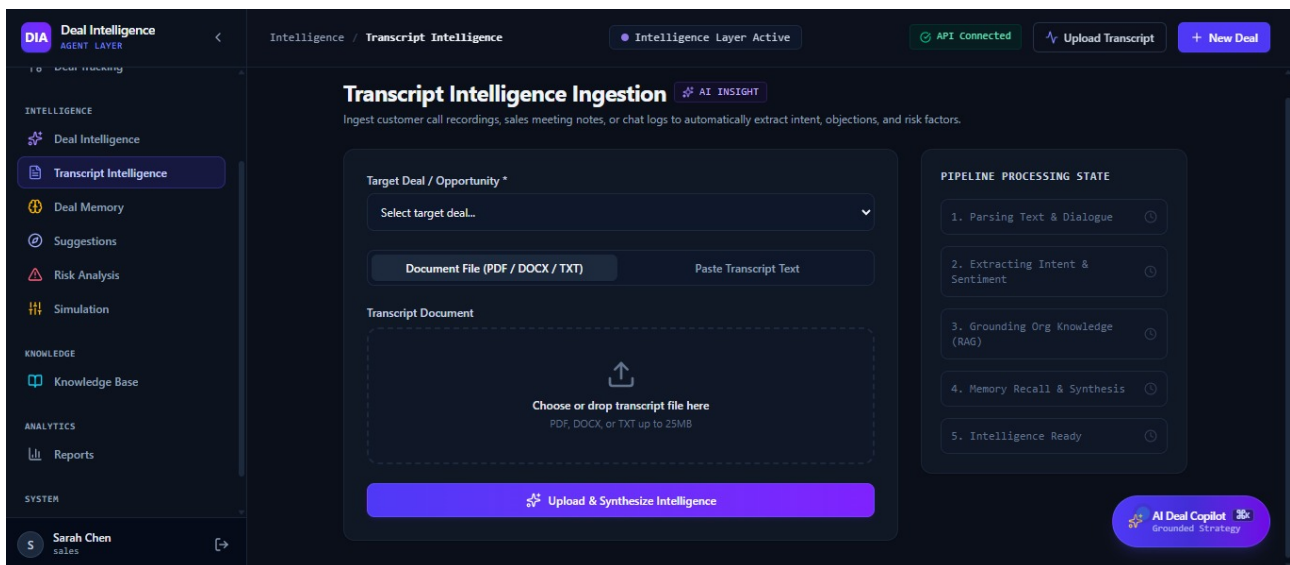


Figure 6 — Transcript Intelligence Ingestion: parsing, intent and sentiment extraction, grounding against organizational knowledge, and memory recall run as a visible five-stage pipeline, not a black box.

Showing the pipeline stages explicitly — parsing, extraction, grounding, recall, synthesis — is a small interface choice with an outsized effect: it turns a multi-second AI process into something a rep can watch and trust, rather than a spinner with no explanation.

## 05 — The Pattern Underneath the Product

Strip away the specific screens, and the same idea repeats at every layer: separate finding a precedent from judging it, separate judging it from deciding what to remember, and never let a confident-sounding output travel without the evidence attached to it. None of the four steps — RAG, Recall, Reflect, Retain — is difficult in isolation. Keeping them distinct, and keeping their evidence visible at every stage, is the actual engineering work.

That is also, ultimately, what separates this from a chatbot layered on top of a CRM. A chatbot answers questions. This system is built to be checked.